ZipVoice: Fast and High-Quality Zero-Shot Text-to-Speech with Flow Matching

Han Zhu, Wei Kang, Zengwei Yao, Liyong Guo, Fangjun Kuang, Zhaoqing Li, Weiji Zhuang, Long Lin, Daniel Povey
Xiaomi Corp., Beijing, China
{zhuhan3,dpovey}@xiaomi.com

***Abstract*—Existing large-scale zero-shot text-to-speech (TTS) models deliver high speech quality but suffer from slow inference speeds due to massive parameters. To address this issue, this paper introduces ZipVoice, a high-quality flow-matching-based zero-shot TTS model with a compact model size and fast inference speed. Key designs include: 1) a Zipformer-based flow-matching decoder to maintain adequate modeling capabilities under constrained size; 2) Average upsampling-based initial speech-text alignment and Zipformer-based text encoder to improve speech intelligibility; 3) A flow distillation method to reduce sampling steps and eliminate the inference overhead associated with classifier-free guidance. Experiments on 100k hours multilingual datasets show that ZipVoice matches state-of-the-art models in speech quality, while being 3 times smaller and up to 30 times faster than a DiT-based flow-matching baseline. Codes, model checkpoints and demo samples are publicly available.[1]**

***Index Terms*—Text-to-speech, zero-shot, flow-matching**

I. Introduction

Text-to-speech (TTS) aims to synthesize natural speech that accurately reflects the provided text. Zero-shot TTS additionally requires the generated speech to mimic the speaker characteristics of a reference audio, enabling flexible adaptation to arbitrary voices. In recent years, zero-shot TTS systems have witnessed substantial progress, driven by the development of generative modeling methods [1]–[7] and the availability of large-scale datasets [8]–[10]. An ideal TTS system should not only be capable of producing high-quality speech but also maintain simplicity and efficiency. Although existing zero-shot TTS models trained on large-scale datasets have achieved impressive speech quality, they typically rely on a large number of parameters to maintain sufficient modeling capacity. Furthermore, most state-of-the-art (SOTA) zero-shot TTS models employ repeated sampling procedures, such as autoregressive (AR) sampling over time [1] or non-autoregressive (NAR) sampling used in diffusion models [11]. The large model size and numerous repeated sampling steps result in slow inference, leading to high deployment costs and limiting the practical applications of zero-shot TTS models.

Recently, flow-matching [12], a variant of diffusion models, has been widely adopted in TTS [2]–[4], [13], [14] due to its reduced sampling steps compared to previous diffusion-based models. A representative flow-matching-based zero-shot TTS system is E2-TTS [3]. E2-TTS addresses text-speech alignment by padding text tokens with filler tokens to match the speech length, and letting the Transformer [15]-based flow-matching decoder to learn the alignment implicitly. This approach significantly simplifies the system architecture and training process by obviating the need for token-level duration prediction. Nevertheless, achieving satisfactory performance with E2-TTS still necessitates a large model size and dozens of sampling steps, which inevitably results in slow inference.

To address the aforementioned challenges, we introduce ZipVoice, a high-quality zero-shot TTS model with a compact model size and fast inference speed. Built upon the flow-matching framework for its simplicity and superior performance, ZipVoice ensure efficient and high-quality speech generation with following designs. Firstly, Zipformer [16] is incorporated as the backbone of flow-matching decoder to ensure adequate model capacity within a limited parameter budget. We demonstrate that the Zipformer architecture, although originally designed for automatic speech recognition (ASR), is also well-suited for TTS tasks. Secondly, since the practice of padding text tokens with filler tokens in E2-TTS results in suboptimal speech-text alignment and compromised intelligibility [4], to ensure the intelligibility of ZipVoice, on the one hand, we propose an average upsampling approach that assumes uniform token durations within a sentence. This straightforward strategy preserves system simplicity by eliminating the need for token-level duration prediction, while maintaining stable alignment between speech and text. On the other hand, a Zipformer-based text encoder is employed to extract better text token representations. Third, flow-matching-based TTS models typically require many sampling steps to generate high-quality speech, and the widely used classifier-free guidance (CFG) [17] strategy further introduces an additional inference pass without condition in each sampling step. To mitigate this, we design a flow distillation method to distill the pre-trained flow-matching model, thereby reducing the number of sampling steps and avoiding the extra computational overhead introduced by CFG.

Experimental results demonstrate that, despite having a model size 3 times smaller and an inference speed up to 30 times faster than a DiT-based flow-matching baseline [4], ZipVoice achieves intelligibility, speaker similarity, and naturalness comparable to existing SOTA zero-shot TTS systems.

II. ZipVoice

In this section, we introduce the proposed zero-shot TTS model, ZipVoice. We first outline the flow-matching method as

[1]https://github.com/k2-fsa/ZipVoice

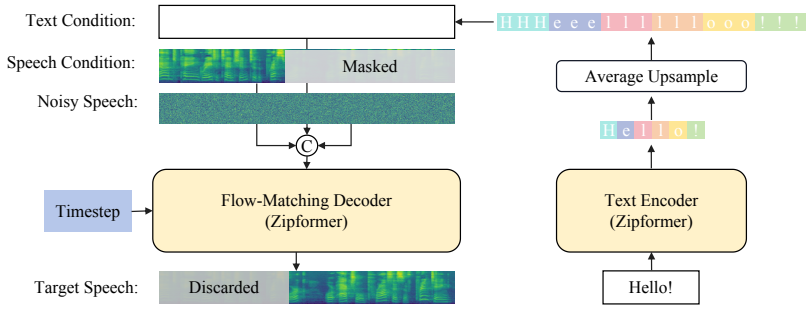

Fig. 1. Illustration of the ZipVoice model architecture.

a preliminary. Next, we provide an overview of ZipVoice. Then, we describe its two Zipformer-based sub-models in detail. We further elaborate on the the average upsampling employed in ZipVoice. Subsequently, we present the speedup version of ZipVoice: ZipVoice-Distill, trained with our flow-distillation method. Finally, we detail the inference strategies of the model.

A. Preliminary: Flow-Matching

ZipVoice is built upon the flow-matching framework [12], specifically conditional flow matching (CFM). We therefore begin with a brief introduction of CFM.

CFM models learn to transform a simple initial distribution p_0 (e.g., standard Gaussian distribution) into a complex data distribution p_1 that approximates the real data distribution q. CFM model is parameterized as a time-dependent vector field $v_t(x_t;\theta), t \in [0,1]$, which can be used to construct a flow ϕ_t that transitions p_0 into p_1. Under the optimal transport form $\phi_t(x) = (1-t)x_0 + tx_1$, the CFM loss can be formulated as:

$$L_{\text{CFM}} = E_{t,q(x_1),p_0(x_0)}||v_t(x_t;\theta) - (x_1 - x_0)||^2 \quad (1)$$

where $x_t = (1-t)x_0 + tx_1$, x_0 is the Gaussian noise, x_1 is the data sample.

After training with the CFM loss, we can generate samples by solving the ordinary differential equation (ODE). Using the Euler solver as an example, starting from an initial sample $x_0 \sim p_0$ (e.g., Gaussian noise), we can iteratively updates the sample towards the target distribution p_1. For a discrete time sequence $0 = t_0 < \cdots < t_k < \cdots < t_K = 1$, the Euler solver updates the sample at each step k as:

$$x_{t_{k+1}} = x_{t_k} + (t_{k+1} - t_k) \cdot v_{t_k}(x_{t_k};\theta). \quad (2)$$

This iteratively transforms x_0 into $x_1 \sim p_1$. The number of function evaluation (NFE) K controls the trade-off between inference speed and sample quality.

Classifier-free guidance (CFG) [17] is widely used to improve the generation quality of flow-matching models. CFG requires dropping the condition with a certain probability during training, and performing linear interpolation of conditional and unconditional predictions during inference:

$$\begin{aligned}\tilde{v}_t(x_t, c, \omega;\theta) =& (1+\omega)v_t(x_t, c;\theta) \\ & - \omega v_t(x_t, \emptyset;\theta)\end{aligned} \quad (3)$$

where c and $\emptyset$ are condition and zero condition, ω is the CFG strength that controls the balance between fidelity and diversity.

B. Overview

The architecture of the ZipVoice model is illustrated in Fig. 1. It consists of two components: a text encoder and a flow-matching decoder, both employing Zipformer as the backbone. We explain the advantage of this design in subsection II-C.

The text sequence to be synthesized is first tokenized into text tokens, denoted as $y = (y_1, y_2, \ldots, y_N)$, where y_i represents the i-th text token and N is the token length. These text tokens are fed into the text encoder and transformed into text features $\hat{y} \in \mathbb{R}^{F \times N}$, where F denotes the text feature dimension. The corresponding speech feature is denoted as $x_1 \in \mathbb{R}^{D \times T}$, where D is the feature dimension and T is the feature length. Then, the average upsampling (detailed in subsection II-D) is applied on the text feature to form the text condition $z \in \mathbb{R}^{F \times T}$.

To enable zero-shot TTS capabilities, we adopt the speech infilling task following [2]. Specifically, a binary temporal mask $m \in \{0,1\}^{D \times T}$, where 1 denotes masked positions, is applied to the speech feature $x_1 \in \mathbb{R}^{D \times T}$. The model is trained to reconstruct $m \odot x_1$ given the speech condition $(1-m) \odot x_1$, the interpolated noisy speech feature $x_t = (1-t)x_0 + tx_1$, and the text condition z. As shown in Fig. 1, these three inputs have the same length and are concatenated along the feature dimension to form the input to the flow-matching decoder.

Then, the CFM objective in (1) can be reformulated as:

$$\begin{aligned}L_{\text{CFM-TTS}} = E_{t,q(x_1),p_0(x_0)} \big\| \big(& v_t(x_t, z, (1-m) \odot x_1;\theta) \\ & - (x_1 - x_0)\big) \odot m \big\|^2\end{aligned} \quad (4)$$

where we mask the training loss with m to discard the losses at the positions of speech conditions.

The standard version of ZipVoice is trained with the above flow-matching loss. We also introduce a speedup version of ZipVoice that is further fine-tuned with flow distillation (detailed in subsection II-E).

After training, we use ODE solver to generate speech features, which is then transformed into waveforms with an additional vocoder model. We also introduce some inference-time strategies to improve the speech quality, which are described in subsection II-F.

C. ZipVoice architecture with Zipformer Backbone

ZipVoice comprises a Zipformer-based text encoder and a Zipformer-based flow-matching decoder, with the latter housing the majority of the model's parameters. Zipformer is an encoder structure designed for automatic speech recognition (ASR). It distinguishes itself from the standard Transformer structures through three key designs: U-Net-like architecture, convolution modules, and attention weight reuse mechanism.

Zipformer is well-suited as the flow-matching decoder backbone for the following reasons. Firstly, the U-Net architecture, which processes feature representations at varying resolutions, is widely recognized as an effective inductive bias in diffusion models [18], [19]. Secondly, convolutional neural networks (CNNs) are proficient in capturing fine-grained local feature patterns, complementing the Transformer's strength in modeling long-range global dependencies. In TTS tasks, the nature of speech leads to strong correlations between adjacent hidden states [20], [21], making CNNs particularly well-suited for enhancing modeling capacity. Finally, Zipformer improves parameter efficiency by reusing attention weights across three modules within a single layer: two self-attention modules and a non-linear attention (NLA) module. This design not only reduces parameter count by sharing projection layers of query and key, but also enhances computational efficiency via attention weight sharing. Therefore, while originally engineered for ASR, these attributes endow Zipformer with ideal compatibility as the backbone of flow-matching decoders.

Although some recent flow-matching models [3], [4] omit dedicated text encoders by feeding text embeddings directly into the flow-matching decoder, we find that a well-designed text encoder can improve speech intelligibility. Our Zipformer-based text encoder retains convolutional modules and attention weight reuse while omitting the U-Net structure, aligning with hybrid CNN-Transformer designs in prior works [22], [23].

D. Speech-Text Alignment with Average Upsampling

A core challenge in NAR-TTS models is how to handle the length discrepancy between text and speech features, i.e., speech-text alignment. NAR-TTS models traditionally require explicit speech-text alignment during training (e.g., ASR-based forced alignment [2] or monotonic alignment search [22]), along with a duration prediction model for inference. This practice complicates the training and can degrade speech naturalness due to inaccurate duration estimates [7].

Some recent NAR-TTS models [3], [5], [24], [25] eliminate the need for explicit alignment. A notable example is E2-TTS [3], which pads text tokens with filter tokens to match the speech length, allowing the model to learn alignment implicitly. This approach significantly simplifies the NAR-TTS architecture by removing the dependency on explicit alignment.

Despite its simplicity, this approach suffers from inaccurate alignment [4] and slow convergence [3]. F5-TTS addresses this by incorporating a ConvNeXt [26] module to refine the padded text condition. ZipVoice instead applies a parameter-free average upsampling strategy to address this issue, under the intuitive assumption of uniform token durations, i.e., each token has identical duration within a sentence. Specifically, when there are N text tokens and T frames speech features frames, the duration of each text token is computed with:

$$d = \left\lfloor \frac{T}{N} \right\rfloor \tag{5}$$

where the floor operation $\lfloor \cdot \rfloor$ ensures the expanded text features do not exceed the speech feature length. Under the practically valid assumption $T \geq N$, the minimum token duration is 1.

Each text embedding is repeated d times and the length of the text feature is expanded from N to $d \cdot N$. If $T > d \cdot N$, the expanded text features is further padded with $T - d \cdot N$ filler embeddings. The final text feature $z \in \mathbb{R}^{F \times T}$ is used as the text condition of the flow-matching decoder.

While this uniform-duration assumption is theoretically simplistic and has a considerable gap between real durations, it provides a reasonable initial text condition for the flow-matching model. Empirically, this straightforward strategy significantly improves alignment accuracy.

E. Speedup ZipVoice with Flow Distillation

In this section, we introduce ZipVoice-Distill, a faster variant of ZipVoice obtained by distilling the ZipVoice model with a flow distillation method. Specifically, we construct a teacher vector field using two-step inference of the teacher model, with CFG applied at each step, then regress the student model's predictions onto this vector field. This enables the student to match the teacher's performance with fewer NFEs and no additional CFG-related evaluations. The flow distillation method is formally described below:

Given a pre-trained TTS model θ^T as the teacher, we initialize the student model θ^S with the parameters of θ^T. Since we want the student model to benefit from the CFG inference while avoiding the additional model evaluation of CFG, we modify the student model to be conditioned on the CFG strength [27]. Specifically, the CFG strength ω is first processed through a Fourier embedding and a linear layer, and then integrated into the model, which is similar to how the time-step is incorporated into the flow-matching decoder.

For any time-step t and input noisy speech x_t, we take two steps with the teacher model to reach a middle time-step t_{mid} and a destination time-step t_{dest}:

$$\begin{aligned} x_{t_{\text{mid}}} &= \Phi(x_t, t, t_{\text{mid}}, c, \omega; \theta^T) \\ x_{t_{\text{dest}}} &= \Phi(x_{t_{\text{mid}}}, t_{\text{mid}}, t_{\text{dest}}, c, \omega; \theta^T) \end{aligned} \tag{6}$$

where the one-step ODE solver Φ is defined as:

$$\Phi(x_t, t, t_{\text{mid}}, c, \omega; \theta^T) = x_t + (t_{\text{mid}} - t)\tilde{v}_t(x_t, c, \omega; \theta^T) \tag{7}$$

where $\tilde{v}_t(x_t, c, \omega; \theta^T)$ is the prediction with CFG as in (3).

The two step sizes, $t_{\text{mid}} - t$ and $t_{\text{dest}} - t_{\text{mid}}$ are uniformly sampled from $[0, \Delta t_{\text{max}}]$. ω is also uniformly sampled between a given interval $[\omega_{\text{min}}, \omega_{\text{max}}]$. We empirically find that dynamically sampling these values yields better performance than using fixed values.

Then, the teacher vector felid is computed as:

$$v^{\mathrm{T}} = \frac{x_{t_{\text{dest}}} - x_t}{t_{\text{dest}} - t} \tag{8}$$

Finally, the flow distillation loss is computed as:

$$L_{\text{FD}} = E_{t,q(x_1),p_0(x_0)} \left\| \left(v_t(x_t, \hat{e}, (1-m) \odot x_1; \theta) - v^T \right) \odot m \right\|^2 \tag{9}$$

Since the teacher prediction in (6) is made with CFG, after flow distillation, the student model can benefit from CFG by simply passing the CFG strength ω as one input of the TTS model, avoiding doubled model evaluation in each step.

After completing flow distillation using the fixed teacher model θ^T, we can conduct a second distillation phase starting from the latest student mode θ^S. The key distinction lies in the construction of the teacher vector field, which now employs the student model's exponential moving averaging (EMA) version $\tilde{\theta}$, which is updated with:

$$\tilde{\theta^S} = (1-\beta)\theta^S + \beta\tilde{\theta} \tag{10}$$

after each training update, where β is the EMA decay factor.

This teacher vector field derived from the continuously evolving student model enables the second distillation phase to refine the student's performance iteratively.

F. Inference Strategy

As a zero-shot TTS model, in addition to the text $y^{synthesis}$ to be synthesized, ZipVoice also requires an audio prompt s^{prompt} and its transcription c^{prompt} mimic its voice.

The sentence duration of the synthesized audio is estimated based on the token length ratio between prompt transcription and the text to be synthesized:

$$T^{synthesis} = T^{prompt} \cdot \frac{|y^{synthesis}|}{|y^{prompt}|} \tag{11}$$

where T^{prompt} is sentence duration of the prompt audio.

The input of the text encoder is the concatenation of tokenized text tokens $y^{synthesis}$ and y^{prompt}. Then, the text condition is constructed with the average upsampled text features. The audio condition is formed by padding the audio prompt to the length $T^{prompt} + T^{synthesis}$. The initial noisy speech is sampled from a standard Gaussian distribution. Finally, the synthesized speech is sampled with an ODE solve.

We design a time-dependent CFG strategy to achieve a better trade-off between speaker similarity and intelligibility: in early NFEs, only the text condition is dropped for unconditional predictions, while in later steps, both text and audio conditions are dropped. This strategy aims to focus more on content-related features in the early steps.

This strategy is exclusively applied to the non-distill version, ZipVoice. For ZipVoice-Distill, the teacher vector field constructed with ZipVoice has incorporated this strategy during flow distillation. Thus, ZipVoice-Distill inherits this property without requiring explicit inference-time adjustments.

III. Related Works

Accelerating TTS models has long been a critical challenge. Early end-to-end TTS models [28], [29] suffered from slow inference due to AR speech generation. FastSpeech [20] addressed this with NAR architectures. But the limited modeling capacity in early NAR models often resulted in blurry or oversmoothed speech [30]. Diffusion models [31] can improve speech quality but rely on many NFEs, making them inefficient. Flow-matching [12] has since been adopted in various works [2], [32] to decrease NFEs. Furthermore, consistency distillation [33], consistency training [33] and ReFlow [34] have also been adopted in some works [35]–[39] to decrease NFEs with a second-stage training or auxiliary training objectives. Parallel to the above efforts, some works [32], [40] accelerate TTS models with efficient model structures. However, most related works focus on small datasets and fail to match SOTA performance across metrics. Our work shows a significant speedup while maintaining SOTA performance across metrics. Moreover, we focus on speeding up NAR-TTS models without explicit speech-text alignment, which is an inherently more challenging and under-explored problem.

IV. Experimental Setup

A. Dataset

We train the ZipVoice model on the large-scale 100k hours Emilia [10] dataset for comparison with existing SOTA zero-shot TTS models. We also train on the small-scale 585 hours LibriTTS [8] dataset for development and ablation experiments. We evaluated our zero-shot TTS model on three widely adopted benchmarks: LibriSpeech-PC [41] test-clean subset [4] with 1127 English samples, Seed-TTS test-en [7] with 1088 English samples from Common Voice [42], and Seed-TTS test-zh with 2020 Chinese samples from DiDiSpeech [43].

B. Model

ZipVoice consists of a Zipformer-based text encoder and a Zipformer-based flow-matching decoder. The text encoder consists of 4 Zipformer layers, each having an encoder dimension of 192, a feedforward dimension of 512. For the flow-matching decoder, it contains 5 stacks of Zipformer encoder layers operating at $[1\times, 2\times, 4\times, 2\times, 1\times]$ downsampling rates and with $[2, 2, 4, 4, 4]$ layers, respectively. Each layer in the flow-matching decoder has a decoder dimension of 512 and a feedforward dimension of 1536. The total number of parameters of ZipVoice models are around 123M.

C. Training

On Emilia/LibriTTS datasets, ZipVoice is trained for 1M/60k updates with the flow-matching objective, followed by 62k/12k updates with the flow distillation objective, employing total batch sizes of 4k/2k seconds. During the training with flow-matching objective, the text condition is dropped with a probability of 20% for CFG. For the speech infilling task, masking lengths are randomly sampled between 70% and 100% of the speech feature length. We use phoneme tokens for Emilia and characters tokens for LibriTTS.

TABLE I

EVALUATION RESULTS OF ZERO-SHOT TTS MODELS TRAINED ON LARGE-SCALE DATASETS. THE BOLDFACE DENOTES THE BEST RESULT. ∗ RESULTS FROM PAPERS. § UNOFFICIAL IMPLEMENTATIONS. † INFERENCE WITH OFFICIAL CHECKPOINTS. ‡ TRAINED BY US WITH OFFICIAL CODES.

Model	Data (hrs)	Params	Objective Metrics									Subjective Metrics	
			LibriSpeech-PC test-clean			Seed-TTS test-en			Seed-TTS test-zh				
			SIM-o ↑	WER ↓	UTMOS ↑	SIM-o ↑	WER ↓	UTMOS ↑	SIM-o ↑	WER ↓	UTMOS ↑	CMOS ↑	SMOS ↑
Ground Truth	-	-	0.690	1.87	4.10	0.734	2.14	3.52	0.755	1.25	2.78	0	3.36
AR Models													
CosyVoice*	170K Multi.	416M	-	-	-	0.609	4.29	-	0.723	3.63	-	-	-
CosyVoice 2*	167K Multi.	618M	-	-	-	0.652	2.57	-	0.748	1.45	-	-	-
Spark-TTS*	102K Multi.	507M	-	-	-	0.584	1.98	-	0.672	**1.20**	-	-	-
NAR Models													
MaskGCT†	100K Emilia	1048M	0.691	2.26	3.91	**0.713**	2.88	3.55	**0.773**	2.40	2.63	-0.08	**4.10**
E2-TTS§ (32 NFE)	100K Emilia	333M	**0.700**	2.49	3.47	0.706	2.32	3.21	0.713	1.91	2.26	-	-
F5-TTS† (32 NFE)	100K Emilia	336M	0.655	1.89	3.89	0.664	1.85	3.72	0.750	1.53	2.93	-0.03	3.76
F5-TTS‡ (32 NFE)	100K Emilia	155M	0.615	2.10	3.84	0.628	1.96	3.66	0.733	1.57	2.93	-	-
ZipVoice (16 NFE)	100K Emilia	123M	0.668	1.64	3.98	0.697	1.70	3.82	0.751	1.40	3.15	**0.17**	3.94
ZipVoice-Distill (8 NFE)	100K Emilia	123M	0.647	1.54	**4.11**	0.670	1.62	**3.91**	0.740	1.34	**3.18**	0.16	3.88
ZipVoice-Distill (4 NFE)	100K Emilia	123M	0.657	**1.51**	4.05	0.679	**1.64**	3.91	0.748	1.39	3.16	0.05	3.84

D. Inference

We use Euler ODE solver for sampling. The pre-trained Vocos [44] vocoder, trained on the LibriTTS dataset, is used to convert the generated speech features into waveforms.

E. Metrics

Three model-based reproducible metrics are used for subjective evaluation. For intelligibility, we use the word error rate (WER) between the transcription of synthesized speech and the input text. We use the Whisper-large-v3 ASR model [45] for Seed-TTS test-en, Paraformer-zh [46] for Seed-TTS test-zh dataset, and the Hubert-based ASR model [47] for LibriSpeech-PC test-clean. For speaker similarity, we use speaker embeddings from a WavLM-based [48] ECAPA-TDNN model [49] to measure the cosine similarity between the original prompt speech and synthesized speech, denoted as SIM-o [2]. For naturalness, we use a neural network-based mean opinion score (MOS) prediction model UTMOS [50].

As for objective evaluation metrics, we use Comparative Mean Opinion Scores (CMOS) and Similarity Mean Opinion Scores (SMOS) to evaluate comparative quality and speaker similarity of the synthesized speech.

V. EXPERIMENTAL RESULTS

A. Comparison with Baseline models on large-scale datasets.

In Table I, we compare ZipVoice with existing SOTA zero-shot TTS models trained on large-scale datasets. Specifically, we benchmark against three AR models (Cosyvoice [51], Cosyvoice 2 [52], and Spark-TTS [53]) and three NAR model (MaskGCT [5], E2-TTS [3], and F5-TTS [4]). For AR models, we report results from [53]. For NAR models, we use the unofficial implementation in [4] for E2-TTS and official implementations for other models. Additionally, we trained a smaller version of F5-TTS with its official codes to illustrate performance degradation due to model size reduction.

Despite its compact model size, ZipVoice delivers performance comparable to other SOTA models that have substantially more parameters. ZipVoice demonstrates clear advantages in WER and UTMOS, while remaining comparable in SIM-o. Although not achieving the best result on all metrics, its overall performance is highly competitive. Comparing ZipVoice and ZipVoice-Distill, flow distillation allows ZipVoice-Distill to prioritize speech quality (WER and UTMOS) with a marginal reduction in speaker similarity (SIM-o).

TABLE II

INFERENCE SPEED COMPARISON OF MODELS.

Model	Params	RTF↓	
		GPU	CPU
F5-TTS (32 NFE)	336M	0.2958	37.284
ZipVoice (16 NFE)	123M	0.0557	9.5529
ZipVoice-Distill (8 NFE)	123M	0.0233	2.4177
ZipVoice-Distill (4 NFE)	123M	**0.0125**	**1.2202**

Since F5-TTS is the fastest model in Table I, we compare the real-time factor (RTF) of F5-TTS and ZipVoice in Table II with same devices and PyTorch versions. All models use the same Vocos [44] vocder, whose RTF is negligible compared with the TTS models. We use 3s prompt speech to generate 10s speech. ZipVoice-Distill (4 NFE) is 23.7 times faster than F5-TTS on the NVIDIA H20 GPU, and 32.6 times faster on a single thread of an Intel(R) Xeon(R) Platinum 8457C CPU. The speed advantage of ZipVoice-Distill comes from three aspects: compact model size, fewer NFEs, and avoiding the additional model inference introduced by CFG. ZipVoice-Distill is close to real-time on a single CPU thread, which can potentially expand the applicable devices of SOTA zero-shot TTS models.

B. Comparison with Baseline models on small-scale datasets.

We also train a model on a LibriTTS, a small-scale dataset, and compare it with other models trained on small-scale datasets. Specifically, we compare with 3 baseline models: VALL-E [1], YourTTS [54] and F5-TTS. We use the unofficial implementation in [55] for VALL-E, and use the official implementation for YourTTS. We train a F5-TTS model on LibriTTS with their official codes for 500k updates.

As shown in Table III, in the small-scale data setting, ZipVoice still shows competitive performance. Among the three baselines, the flow-matching-based model F5-TTS demonstrates

TABLE III
EVALUATION RESULTS OF ZERO-SHOT TTS MODELS TRAINED ON SMALL-SCALE DATASETS. THE BOLDFACE DENOTES THE BEST RESULT. § UNOFFICIAL IMPLEMENTATIONS. † INFERENCE WITH OFFICIAL CHECKPOINTS. ‡ TRAINED BY US WITH OFFICIAL CODES.

Model	Data (hrs)	Params	LibriSpeech-PC test-clean		
			SIM-o↑	WER↓	UTMOS↑
Ground-truth	–	–	0.690	1.87	4.10
VALL-E§	6K Libri-light	367M	0.369	6.20	3.01
YourTTS†	474 Multi.	87M	0.462	6.37	3.68
F5-TTS (32 NFE)‡	555 LibriTTS	158M	0.584	1.78	4.14
ZipVoice (8 NFE)	555 LibriTTS	123M	**0.610**	1.69	4.16
ZipVoice-Distill (4 NFE)	555 LibriTTS	123M	0.606	**1.68**	**4.22**

the strongest performance, demonstrating the robustness of flow-matching model. Although with smaller model size and fewer training updates, ZipVoice and ZipVoice-Distill are consistently better than F5-TTS on all metrics.

C. Ablation experiments on the ZipVoice structure

TABLE IV
ABLATION STUDIES ON THE ZIPVOICE ARCHITECTURE

Model architecture	LibriSpeech-PC test-clean		
	SIM-o↑	WER↓	UTMOS↑
ZipVoice	0.610	1.69	4.16
w/o text encoder	0.597	2.04	4.15
w/o average upsample	0.513	20.19	3.89
+ w/ ConvNext	0.524	15.49	3.80

In this section, we show the impact of two architecture designs in ZipVoice: the text encoder and the average upsampling strategy. As shown in Table IV, both designs are important in ensuring high speech intelligibility (low WER). Notably, removing the average upsampling leads to a substantial performance degradation. We also tested the ConvNext-based text condition refinement method proposed in [4]. While the method also reduces WER to some extent, the average upsampling strategy still shows a clear performance advantage.

D. Ablation experiments on the Zipformer backbone

TABLE V
ABLATION STUDIES ON THE BACKBONE STRUCTURE.

Backbone Structure	LibriSpeech-PC test-clean		
	SIM-o↑	WER↓	UTMOS↑
Zipformer	0.610	1.69	4.16
w/o convolution	0.586	9.79	4.01
w/o downsampling	0.557	3.70	4.02
w/o downsample & bypass	0.562	6.35	4.08
w/o bypass	0.179	98.89	1.25
w/o NLA	0.548	1.83	3.99
w/o share attention weight	0.595	1.75	4.18

We conduct detailed ablation experiments to validate the effectiveness of using Zipformer as the backbone of the flow-matching decoder. As shown in Table V, convolution modules are important in assuring high intelligibility. The U-Net-like downsampling and bypass are consistently helpful in all metrics, showing the effectiveness of the U-Net-like inductive bias on the flow-matching-based TTS model. Notably, removing downsampling but keeping the bypass leads to an architecture similar to the flat Unet-style linked Transformer architecture in [3], which has better results than the model without both downsampling and bypass. And removing bypass connections while retaining downsampling leads to severely degraded results, highlighting the importance of cross-resolution bypass connections in U-net style downsampling structure. Finally, we show the impact of re-using the attention weight. By re-using the attention weight in the NLA module, the performance is consistently improved in all metrics. And the design of sharing attention weight in two self-attention module leads to similar results with the baseline but has improved efficiency.

E. Experiments with Different Distillation Methods

TABLE VI
ABLATION STUDY OF DISTILLATION METHODS WITH DIFFERENT NFE. THE BOLDFACE DENOTES BEST RESULTS IN EACH NFE.

Distillation Method	NFE	LibriSpeech-PC test-clean		
		SIM-o↑	WER↓	UTMOS↑
Model w/o distillation	1	0.171	92.17	1.30
	2	0.475	15.00	1.83
	4	**0.634**	2.11	3.84
Consistency distillation	1	0.441	20.72	1.53
	2	0.571	4.06	3.08
	4	0.568	1.97	3.30
ReFlow	1	**0.512**	**17.40**	**1.90**
	2	**0.601**	5.36	3.39
	4	0.608	2.56	3.92
Our flow distillation	1	0.398	18.81	1.58
	2	0.588	**2.33**	**3.73**
	4	0.606	**1.68**	**4.22**

In this section, we demonstrate the effectiveness of the proposed flow distillation method. We compare our flow distillation method with two popular acceleration methods of flow-matching models: consistency distillation [33] and ReFlow [34]. We train all models with the same number of updates. As shown in Table VI, consistency distillation effectively improves the performance at 1 and 2 NFEs, but fails to improve performance at 4 NFEs. However, achieving satisfactory performance with just 1 or 2 NFEs is challenging for ZipVoice due to the absence of explicit token-level duration in the text condition. Consequently, consistency distillation is not applicable in this context. ReFlow [34] improves UTMOS across all NFE settings, but leads to WER degradation with with NFEs larger than 1. In contrast, our flow distillation method can consistently improves performance across different NFEs and achieves the best result at 4 NFEs.

VI. CONCLUSION

In this paper, we propose ZipVoice, a fast and high-quality flow-matching-based zero-shot TTS model. Leveraging a Zipformer-based backbone for the flow-matching decoder, ZipVoice achieves parameter efficiency while maintaining strong modeling capabilities. An average upsampling strategy,

combined with a Zipformer text encoder, ensures stable speech-text alignment and intelligibility. In addition, a flow distillation method is proposed to reduce the NFEs, significantly accelerating inference. Experimental results show that ZipVoice matches SOTA zero-shot TTS models in speech quality but with fewer parameters and faster inference speed.

REFERENCES

[1] C. Wang, S. Chen, Y. Wu, Z. Zhang, L. Zhou, S. Liu, Z. Chen, Y. Liu, H. Wang, J. Li *et al.*, "Neural codec language models are zero-shot text to speech synthesizers," *arXiv preprint arXiv:2301.02111*, 2023.

[2] M. Le, A. Vyas, B. Shi, B. Karrer, L. Sari, R. Moritz, M. Williamson, V. Manohar, Y. Adi, J. Mahadeokar *et al.*, "Voicebox: Text-guided multilingual universal speech generation at scale," *Advances in neural information processing systems*, vol. 36, pp. 14 005–14 034, 2023.

[3] S. E. Eskimez, X. Wang, M. Thakker, C. Li, C.-H. Tsai, Z. Xiao, H. Yang, Z. Zhu, M. Tang, X. Tan *et al.*, "E2 tts: Embarrassingly easy fully non-autoregressive zero-shot tts," *arXiv preprint arXiv:2406.18009*, 2024.

[4] Y. Chen, Z. Niu, Z. Ma, K. Deng, C. Wang, J. Zhao, K. Yu, and X. Chen, "F5-tts: A fairytaler that fakes fluent and faithful speech with flow matching," *arXiv preprint arXiv:2410.06885*, 2024.

[5] Y. Wang, H. Zhan, L. Liu, R. Zeng, H. Guo, J. Zheng, Q. Zhang, X. Zhang, S. Zhang, and Z. Wu, "Maskgct: Zero-shot text-to-speech with masked generative codec transformer," *arXiv preprint arXiv:2409.00750*, 2024.

[6] H.-H. Guo, Y. Hu, K. Liu, F.-Y. Shen, X. Tang, Y.-C. Wu, F.-L. Xie, K. Xie, and K.-T. Xu, "Fireredtts: A foundation text-to-speech framework for industry-level generative speech applications," *arXiv preprint arXiv:2409.03283*, 2024.

[7] P. Anastassiou, J. Chen, J. Chen, Y. Chen, Z. Chen, Z. Chen, J. Cong, L. Deng, C. Ding, L. Gao *et al.*, "Seed-tts: A family of high-quality versatile speech generation models," *arXiv preprint arXiv:2406.02430*, 2024.

[8] H. Zen, V. Dang, R. Clark, Y. Zhang, R. J. Weiss, Y. Jia, Z. Chen, and Y. Wu, "Libritts: A corpus derived from librispeech for text-to-speech," in *Proc. Interspeech 2019*, 2019, pp. 1526–1530.

[9] W. Kang, X. Yang, Z. Yao, F. Kuang, Y. Yang, L. Guo, L. Lin, and D. Povey, "Libriheavy: A 50,000 hours asr corpus with punctuation casing and context," in *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2024, pp. 10 991–10 995.

[10] H. He, Z. Shang, C. Wang, X. Li, Y. Gu, H. Hua, L. Liu, C. Yang, J. Li, P. Shi *et al.*, "Emilia: An extensive, multilingual, and diverse speech dataset for large-scale speech generation," in *2024 IEEE Spoken Language Technology Workshop (SLT)*. IEEE, 2024, pp. 885–890.

[11] K. Shen, Z. Ju, X. Tan, E. Liu, Y. Leng, L. He, T. Qin, S. Zhao, and J. Bian, "Naturalspeech 2: Latent diffusion models are natural and zero-shot speech and singing synthesizers," in *ICLR*, 2024.

[12] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *The Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=PqvMRDCJT9t

[13] X. Li, Z. Shang, H. Hua, P. Shi, C. Yang, L. Wang, and P. Zhang, "Sf-speech: Straightened flow for zero-shot voice clone," *IEEE Transactions on Audio, Speech and Language Processing*, 2025.

[14] S. Kim, K. Shih, J. F. Santos, E. Bakhturina, M. Desta, R. Valle, S. Yoon, B. Catanzaro *et al.*, "P-flow: A fast and data-efficient zero-shot tts through speech prompting," *Advances in Neural Information Processing Systems*, vol. 36, pp. 74 213–74 228, 2023.

[15] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[16] Z. Yao, L. Guo, X. Yang, W. Kang, F. Kuang, Y. Yang, Z. Jin, L. Lin, and D. Povey, "Zipformer: A faster and better encoder for automatic speech recognition," in *ICLR*, 2024.

[17] J. Ho and T. Salimans, "Classifier-free diffusion guidance," in *NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications*, 2021. [Online]. Available: https://openreview.net/forum?id=qw8AKxfYbI

[18] C. Si, Z. Huang, Y. Jiang, and Z. Liu, "Freeu: Free lunch in diffusion u-net," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 4733–4743.

[19] Y. Tian, Z. Tu, H. Chen, J. Hu, C. Xu, and Y. Wang, "U-dits: Downsample tokens in u-shaped diffusion transformers," in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.

[20] Y. Ren, Y. Ruan, X. Tan, T. Qin, S. Zhao, Z. Zhao, and T.-Y. Liu, "Fastspeech: Fast, robust and controllable text to speech," *Advances in neural information processing systems*, vol. 32, 2019.

[21] A. Gulati, J. Qin, C.-C. Chiu, N. Parmar, Y. Zhang, J. Yu, W. Han, S. Wang, Z. Zhang, Y. Wu *et al.*, "Conformer: Convolution-augmented transformer for speech recognition," in *Proc. Interspeech 2020*, 2020, pp. 5036–5040.

[22] J. Kim, S. Kim, J. Kong, and S. Yoon, "Glow-tts: A generative flow for text-to-speech via monotonic alignment search," *Advances in Neural Information Processing Systems*, vol. 33, pp. 8067–8077, 2020.

[23] C. Miao, S. Liang, M. Chen, J. Ma, S. Wang, and J. Xiao, "Flow-tts: A non-autoregressive network for text to speech based on flow," in *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2020, pp. 7209–7213.

[24] D. Yang, D. Wang, H. Guo, X. Chen, X. Wu, and H. Meng, "Simple-speech: Towards simple and efficient text-to-speech with scalar latent transformer diffusion models," *Proc. INTERSPEECH*, 2024.

[25] K. Lee, D. W. Kim, J. Kim, and J. Cho, "Ditto-tts: Efficient and scalable zero-shot text-to-speech with diffusion transformer," *arXiv preprint arXiv:2406.11427*, 2024.

[26] S. Woo, S. Debnath, R. Hu, X. Chen, Z. Liu, I. S. Kweon, and S. Xie, "Convnext v2: Co-designing and scaling convnets with masked autoencoders," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2023, pp. 16 133–16 142.

[27] C. Meng, R. Rombach, R. Gao, D. Kingma, S. Ermon, J. Ho, and T. Salimans, "On distillation of guided diffusion models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 14 297–14 306.

[28] Y. Wang, R. Skerry-Ryan, D. Stanton, Y. Wu, R. J. Weiss, N. Jaitly, Z. Yang, Y. Xiao, Z. Chen, S. Bengio *et al.*, "Tacotron: Towards end-to-end speech synthesis," in *Proc. Interspeech 2017*, 2017, pp. 4006–4010.

[29] J. Shen, R. Pang, R. J. Weiss, M. Schuster, N. Jaitly, Z. Yang, Z. Chen, Y. Zhang, Y. Wang, R. Skerrv-Ryan *et al.*, "Natural tts synthesis by conditioning wavenet on mel spectrogram predictions," in *2018 IEEE international conference on acoustics, speech and signal processing (ICASSP)*. IEEE, 2018, pp. 4779–4783.

[30] Y. Ren, X. Tan, T. Qin, Z. Zhao, and T.-Y. Liu, "Revisiting over-smoothness in text to speech," *arXiv preprint arXiv:2202.13066*, 2022.

[31] V. Popov, I. Vovk, V. Gogoryan, T. Sadekova, and M. Kudinov, "Grad-tts: A diffusion probabilistic model for text-to-speech," in *International Conference on Machine Learning*. PMLR, 2021, pp. 8599–8608.

[32] S. Mehta, R. Tu, J. Beskow, É. Székely, and G. E. Henter, "Matcha-tts: A fast tts architecture with conditional flow matching," in *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2024, pp. 11 341–11 345.

[33] Y. Song, P. Dhariwal, M. Chen, and I. Sutskever, "Consistency models," in *International Conference on Machine Learning*. PMLR, 2023, pp. 32 211–32 252.

[34] X. Liu, C. Gong *et al.*, "Flow straight and fast: Learning to generate and transfer data with rectified flow," in *The Eleventh International Conference on Learning Representations*, 2023.

[35] Z. Ye, W. Xue, X. Tan, J. Chen, Q. Liu, and Y. Guo, "Comospeech: One-step speech and singing voice synthesis via consistency model," in *Proceedings of the 31st ACM International Conference on Multimedia*, 2023, pp. 1831–1839.

[36] W. Guan, Q. Su, H. Zhou, S. Miao, X. Xie, L. Li, and Q. Hong, "Reflow-tts: A rectified flow model for high-fidelity text-to-speech," in *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2024, pp. 10 501–10 505.

[37] Y. Guo, C. Du, Z. Ma, X. Chen, and K. Yu, "Voiceflow: Efficient text-to-speech with rectified flow matching," in *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2024, pp. 11 121–11 125.

[38] Z. Ye, Z. Ju, H. Liu, X. Tan, J. Chen, Y. Lu, P. Sun, J. Pan, W. Bian, S. He *et al.*, "Flashspeech: Efficient zero-shot speech synthesis," in *Proceedings of the 32nd ACM International Conference on Multimedia*, 2024, pp. 6998–7007.

[39] K. Wang, W. Guan, S. Lu, J. Yao, L. Li, and Q. Hong, "Slimspeech: Lightweight and efficient text-to-speech with slim rectified flow," in *ICASSP 2025-2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2025, pp. 1–5.

[40] R. Luo, X. Tan, R. Wang, T. Qin, J. Li, S. Zhao, E. Chen, and T.-Y. Liu, "Lightspeech: Lightweight and fast text to speech with neural architecture search," in *ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2021, pp. 5699–5703.

[41] A. Meister, M. Novikov, N. Karpov, E. Bakhturina, V. Lavrukhin, and B. Ginsburg, "Librispeech-pc: Benchmark for evaluation of punctuation and capitalization capabilities of end-to-end asr models," in *2023 IEEE automatic speech recognition and understanding workshop (ASRU)*. IEEE, 2023, pp. 1–7.

[42] R. Ardila, M. Branson, K. Davis, M. Kohler, J. Meyer, M. Henretty, R. Morais, L. Saunders, F. Tyers, and G. Weber, "Common voice: A massively-multilingual speech corpus," in *Proceedings of the Twelfth Language Resources and Evaluation Conference*, 2020, pp. 4218–4222.

[43] T. Guo, C. Wen, D. Jiang, N. Luo, R. Zhang, S. Zhao, W. Li, C. Gong, W. Zou, K. Han *et al.*, "Didispeech: A large scale mandarin speech corpus," in *ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2021, pp. 6968–6972.

[44] H. Siuzdak, "Vocos: Closing the gap between time-domain and fourier-based neural vocoders for high-quality audio synthesis," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=vY9nzQmQBw

[45] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *International conference on machine learning*. PMLR, 2023, pp. 28 492–28 518.

[46] Z. Gao, S. Zhang, I. McLoughlin, and Z. Yan, "Paraformer: Fast and accurate parallel transformer for non-autoregressive end-to-end speech recognition," in *Proc. Interspeech 2022*, 2022, pp. 2063–2067.

[47] W.-N. Hsu, B. Bolte, Y.-H. H. Tsai, K. Lakhotia, R. Salakhutdinov, and A. Mohamed, "Hubert: Self-supervised speech representation learning by masked prediction of hidden units," *IEEE/ACM transactions on audio, speech, and language processing*, vol. 29, pp. 3451–3460, 2021.

[48] S. Chen, C. Wang, Z. Chen, Y. Wu, S. Liu, Z. Chen, J. Li, N. Kanda, T. Yoshioka, X. Xiao *et al.*, "Wavlm: Large-scale self-supervised pre-training for full stack speech processing," *IEEE Journal of Selected Topics in Signal Processing*, vol. 16, no. 6, pp. 1505–1518, 2022.

[49] B. Desplanques, J. Thienpondt, and K. Demuynck, "Ecapa-tdnn: Emphasized channel attention, propagation and aggregation in tdnn based speaker verification," in *Proc. Interspeech 2020*, 2020, pp. 3830–3834.

[50] T. Saeki, D. Xin, W. Nakata, T. Koriyama, S. Takamichi, and H. Saruwatari, "Utmos: Utokyo-sarulab system for voicemos challenge 2022," *Interspeech 2022*, 2022.

[51] Z. Du, Q. Chen, S. Zhang, K. Hu, H. Lu, Y. Yang, H. Hu, S. Zheng, Y. Gu, Z. Ma *et al.*, "Cosyvoice: A scalable multilingual zero-shot text-to-speech synthesizer based on supervised semantic tokens," *arXiv preprint arXiv:2407.05407*, 2024.

[52] Z. Du, Y. Wang, Q. Chen, X. Shi, X. Lv, T. Zhao, Z. Gao, Y. Yang, C. Gao, H. Wang *et al.*, "Cosyvoice 2: Scalable streaming speech synthesis with large language models," *arXiv preprint arXiv:2412.10117*, 2024.

[53] X. Wang, M. Jiang, Z. Ma, Z. Zhang, S. Liu, L. Li, Z. Liang, Q. Zheng, R. Wang, X. Feng *et al.*, "Spark-tts: An efficient llm-based text-to-speech model with single-stream decoupled speech tokens," *arXiv preprint arXiv:2503.01710*, 2025.

[54] E. Casanova, J. Weber, C. D. Shulby, A. C. Junior, E. Gölge, and M. A. Ponti, "Yourtts: Towards zero-shot multi-speaker tts and zero-shot voice conversion for everyone," in *International conference on machine learning*. PMLR, 2022, pp. 2709–2720.

[55] X. Zhang, L. Xue, Y. Gu, Y. Wang, J. Li, H. He, C. Wang, S. Liu, X. Chen, J. Zhang *et al.*, "Amphion: an open-source audio, music, and speech generation toolkit," in *2024 IEEE Spoken Language Technology Workshop (SLT)*. IEEE, 2024, pp. 879–884.